Software-in-the-Loop Firmware Simulation Infrastructure

Kevin Litvin | SFU Formula SAE Electric (Team Phantom)
github.com/kevinl03 | klitvin101@gmail.com

Project Purpose

Team Phantom is SFU's Formula SAE Electric racing team (80+ students). I joined as a controls member in 2022, led the Vehicle Control Unit (VCU) project in 2023–2024, and served as Senior Infrastructure Engineer in 2024–2025. This document describes the software-in-the-loop (SIL) simulation framework I designed and built to enable the team to develop, test, and validate vehicle control firmware entirely in software, without requiring physical hardware.

The Problem

Our existing testing framework simulated the FSAE electric vehicle's control systems (throttle response, battery thermal behavior, tire traction) but it required a physical microcontroller connected via USB to communicate with the firmware under test. This created several compounding problems:

- **Hardware bottleneck:** Only one or two dev boards existed on the team. During a chip shortage, replacements took months. Five developers were blocked by two pieces of hardware.
- **Ambiguous failures:** Failures could originate in firmware, the test harness, USB communication, or the microcontroller itself. Isolating software bugs from hardware issues consumed significant debugging time.
- **No automation:** Tests required a physical USB connection, so there was no path to CI/CD. Every run was manual and regressions could survive undetected for weeks.

Root-Cause Analysis

The firmware's control logic was pure C code. It didn't inherently need a physical microcontroller to execute. However, all hardware peripheral communication (SPI, ADC, DAC, GPIO, CAN) was tightly coupled to the microcontroller's Hardware Abstraction Layer (HAL). To run the firmware on a developer's machine, every HAL call needed to be redirected to a logging and simulation system instead of real hardware registers. This meant rewriting the HAL so it compiled differently depending on the target platform: the original microcontroller drivers for production, the Win32 API for Windows, or POSIX for macOS and Linux. If the HAL redirection, the FreeRTOS runtime, and the USB communication channel could all be replaced with software equivalents, the entire pipeline could run without hardware.

What I Built

The solution had three layers:

1. FreeRTOS Kernel Emulation Layer (C/C++). I wrote a shim layer that reimplemented the FreeRTOS API (task creation, scheduling, queues, semaphores, timers) on top of POSIX threads and Win32 threading primitives. The firmware compiled against this layer instead of the real kernel, producing a native executable that ran identically on a developer's machine.

2. Interprocess Communication Pipeline (C, Named Pipes). I replaced the USB channel with named-pipe IPC. The simulation framework and firmware ran as separate processes communicating through OS-level pipes, preserving real-time bidirectional data flow while being entirely software-based.

3. CI/CD and Cloud Infrastructure (Python, GitHub Actions, Docker, GCP). With hardware removed from the loop, the SIL framework ran inside a Docker container, standardizing the build environment so every pull request triggered identical automated firmware validation. Test results, simulation plots, and run metadata were uploaded via Google Cloud Platform APIs to a shared Team Phantom Google Drive, giving all 100+ team members a unified location to reference validation data without needing local access to the toolchain.

The architecture was firmware-agnostic by design. The IPC pipeline only needed a named pipe endpoint, so the same framework worked for both the VCU and BMS firmware without modification.

Results

- **60% reduction in test-cycle time**, from 10 weeks down to 4 weeks. Developers could run firmware tests on their laptops instead of waiting for scheduled hardware sessions.
- **Hardware dependency eliminated** for the 5-person software sub-team. The microcontroller shortage became irrelevant overnight.
- **Automated regression testing** via CI/CD caught firmware regressions at the pull-request stage, before hardware integration.

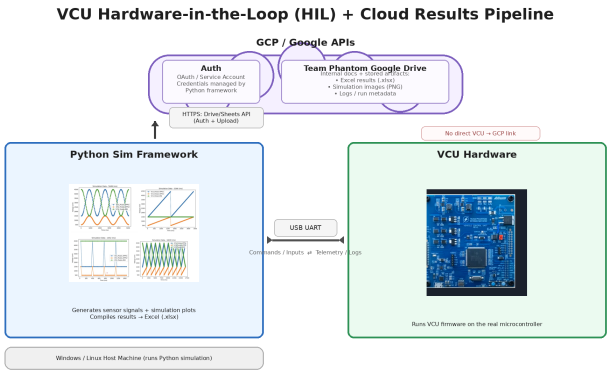


Figure 1: Existing HIL pipeline: Python simulation communicates with real VCU hardware over USB UART, requiring a physical microcontroller for every test run.

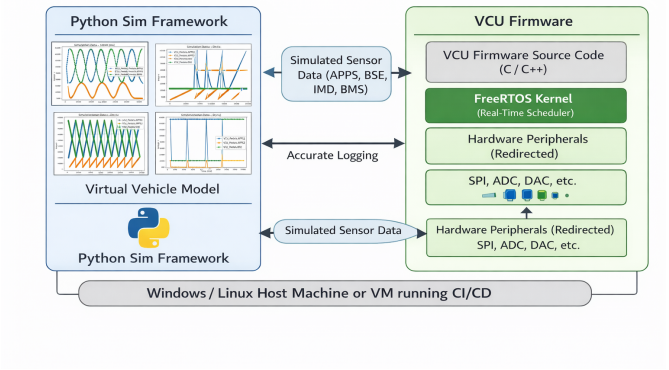


Figure 2: SIL replacement: Python simulation and VCU firmware both run on the host machine with redirected virtual drivers, enabling automated CI/CD.

- **Cross-platform development**: the emulation layer ran on both macOS and Windows, matching the team's mixed environments.

My Role and Source Code

I owned this project end-to-end: problem identification, architecture, implementation, and deployment. I collaborated with firmware developers to verify scheduling fidelity and with the electronics sub-team to validate simulation accuracy against hardware results.

Source code is available under the SFU Phantom organization on GitHub:

- **VCU SIL Framework** (FreeRTOS kernel emulation, Windows/Linux port): github.com/sfuphantom/VCU-SIL-Framework-Setup
- **Testing Framework** (simulation generation, IPC pipeline in C, Google Cloud communication): github.com/sfuphantom/TestingFramework
- **VCU Firmware** (target firmware for the TMS570 Vehicle Control Unit): github.com/sfuphantom/vcu-fw

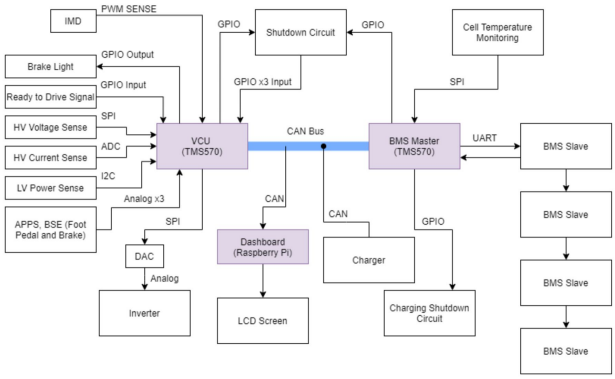


Figure 3: EV-VCU system architecture: VCU and BMS communicate with sensors, actuators, and dashboard over CAN bus, SPI, I2C, GPIO, and analog interfaces.

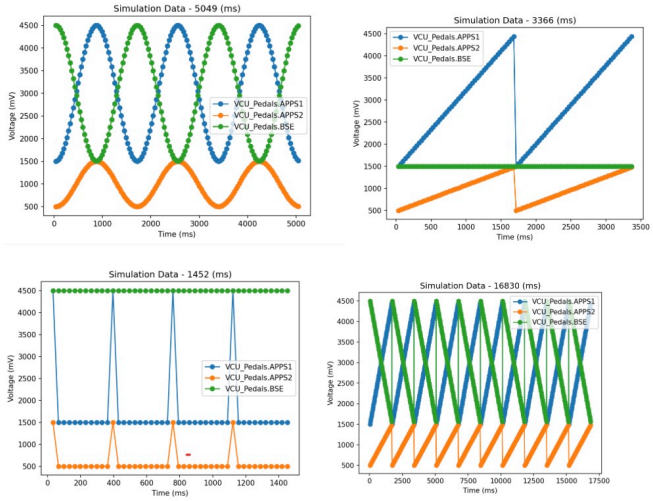


Figure 4: Simulation output: real-time pedal sensor data (APPS1, APPS2, BSE) across multiple test scenarios consumed by the emulated VCU firmware.